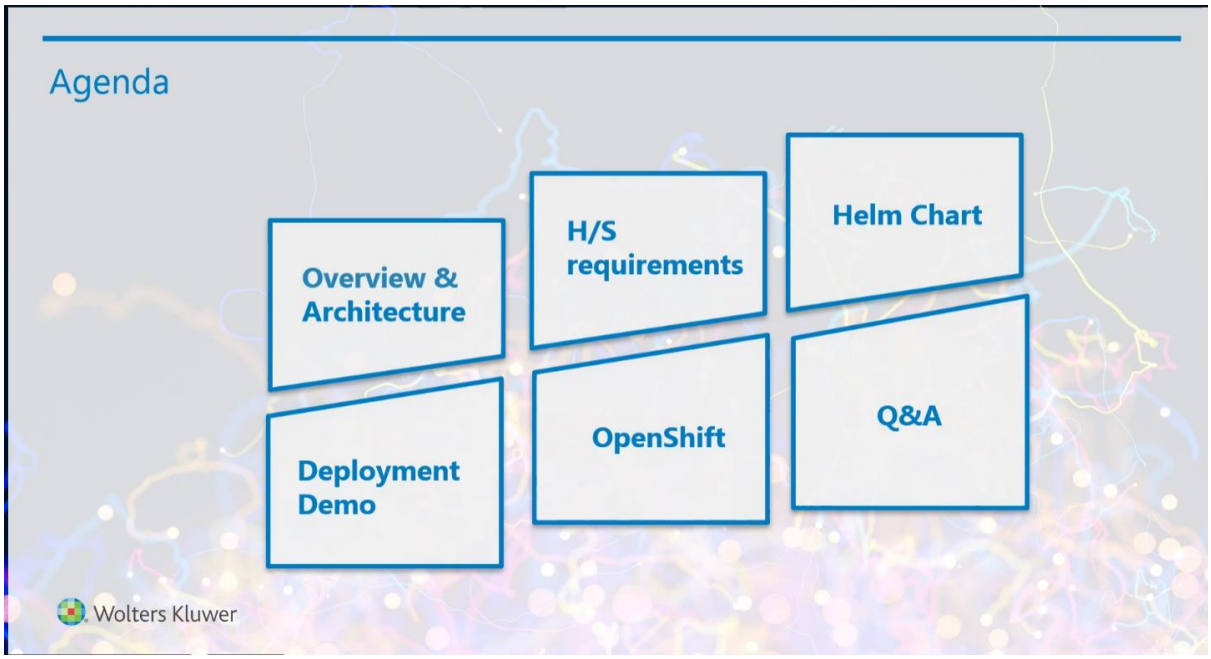
Day 5
14-06-2024

Setup OneSumx platform with Kubernetes.

Trainer: Nagy, Gábor Albert



Overview & Architecture

When to deploy OneSumX on K8S?

- ✓ Investment in existing Infrastructure / Subscription
- ✓ Preparing for scalable solution
- ✓ Go-to-option for cloud

K8S used for OneSumX SaaS because:

- ✓ Improved Disaster Recovery
- ✓ Security – e.g. AppGateway support
- ✓ Leveraging AKS tools

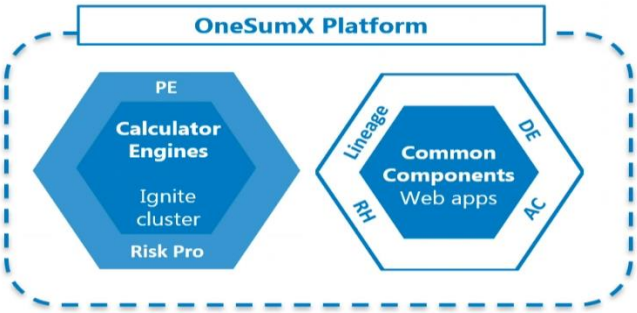
Packaged by Helm

- ✓ Easily configurable from command line
- ✓ Different deployment scenarios
- ✓ Supports upgrade

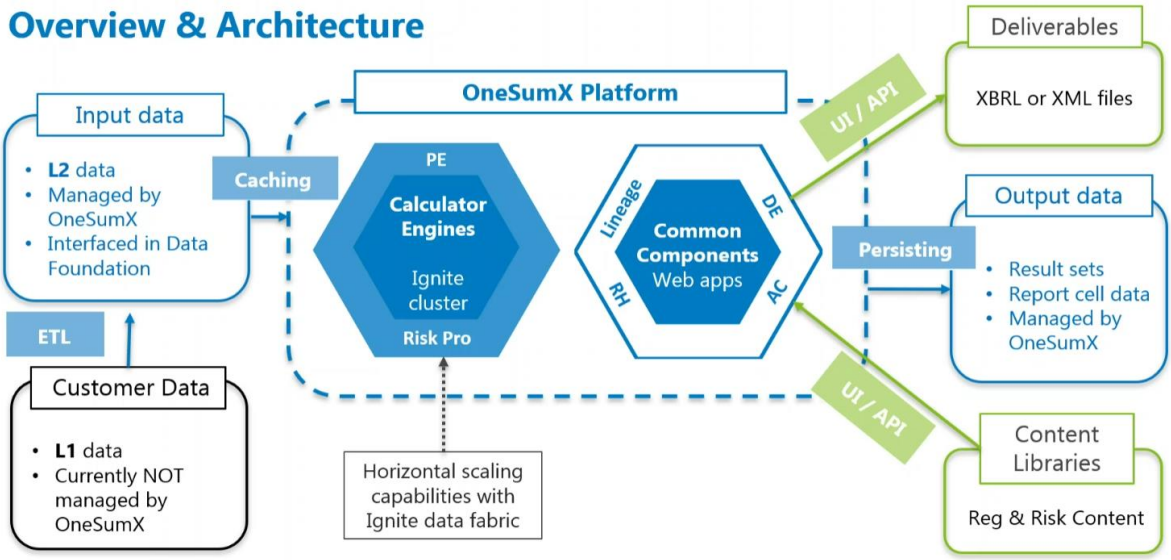
Could and On-Premise

- ✓ Tested and certified for AKS and OpenShift
- ✓ Documented these two K8S platforms
- ✓ Supports cloud or on-premise deployment

Overview & Architecture



Overview & Architecture

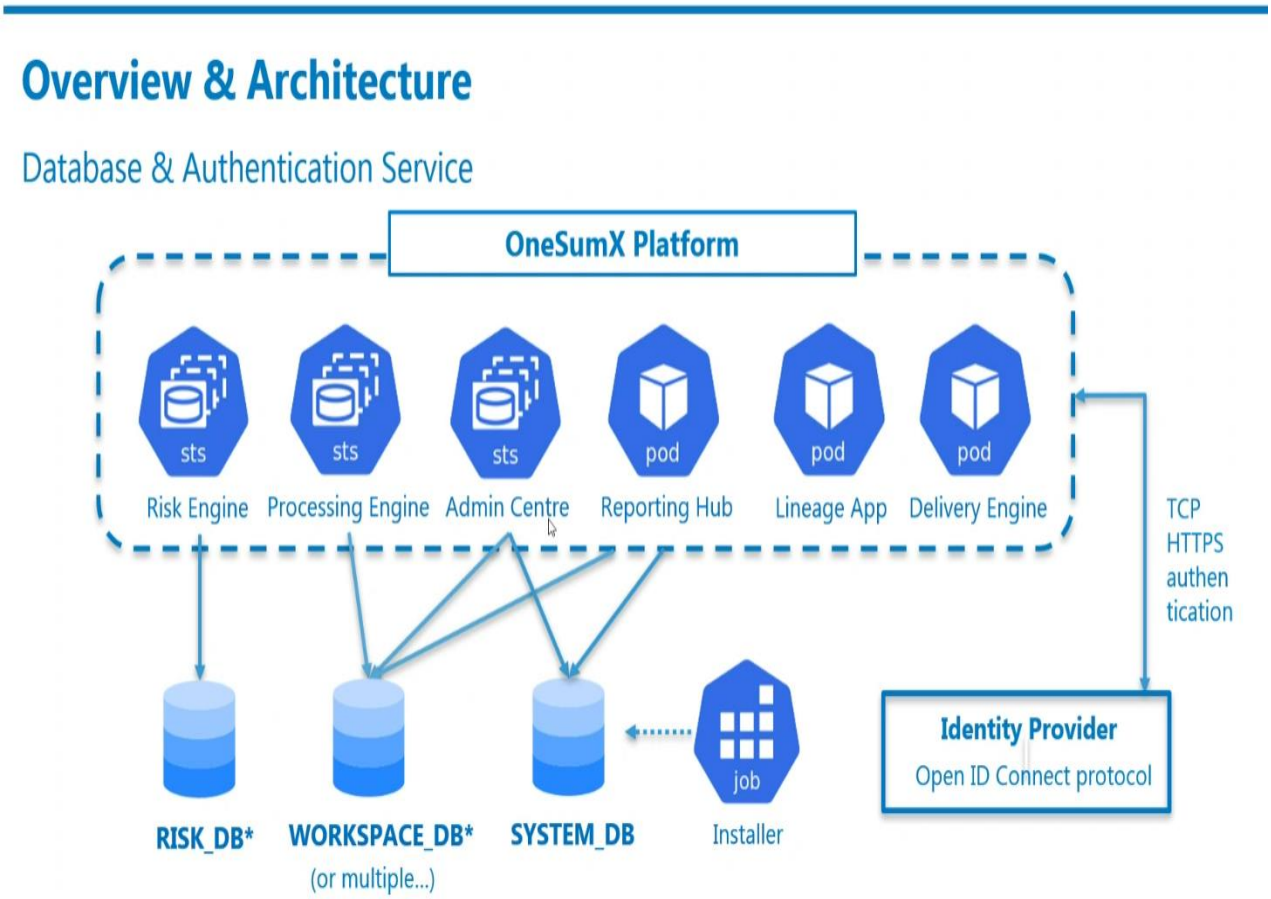


- L1 data is from banking side and L2 loading the data from first table through Onesumx and

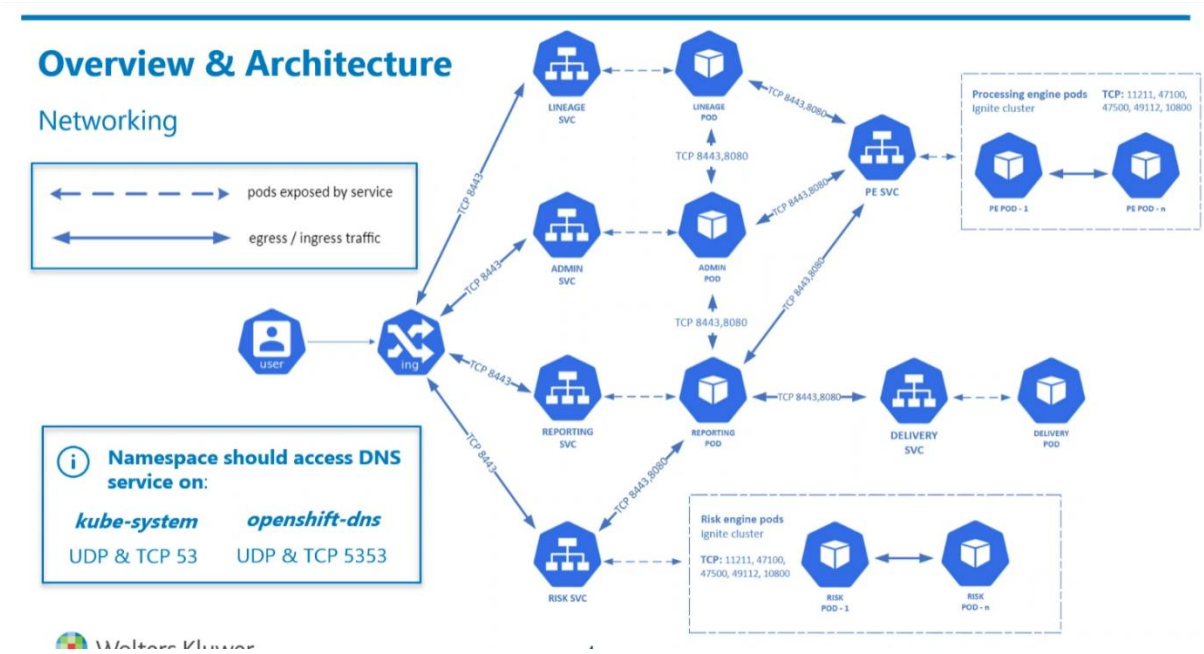
caching it. Cache turn into files, that is lineage. Once the data is persisting then we are saving it to database.

- Calculation engine will be horizontally scalable
- Dliverble should be in XBRL or XML file

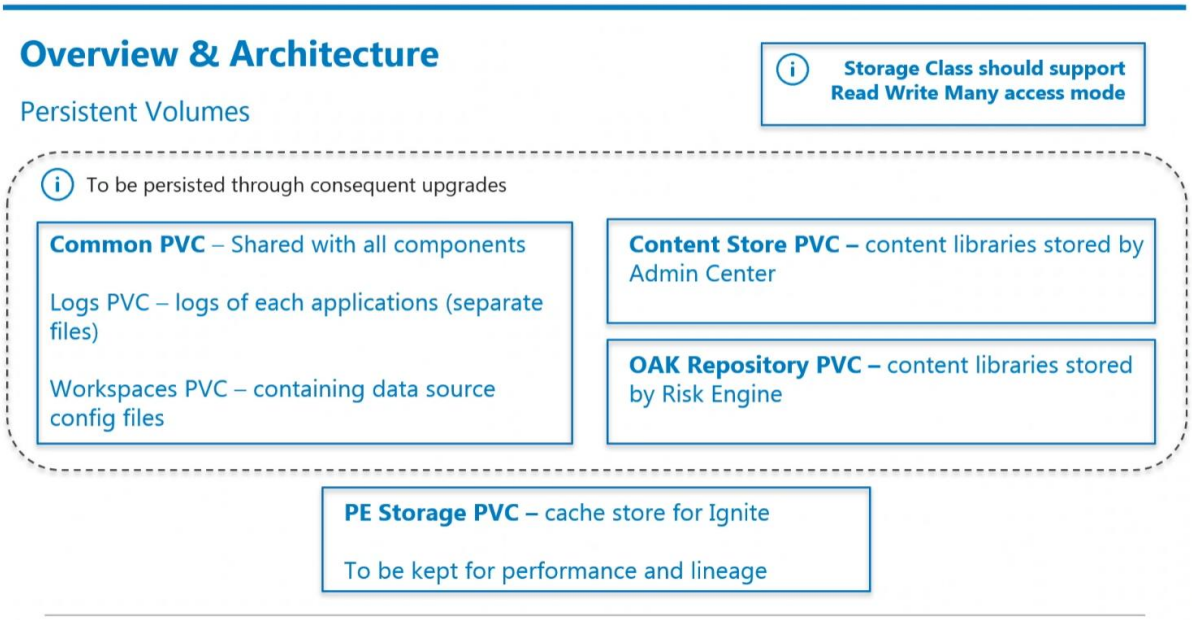
Overview & Architecture



I above screenshot Processing engine is stateful set .



Persistent volumes



Common pvc : Logs of each application are stored in common Pvc

Content store: Content libraries stored in content store

PVC : Caches are stored in PVC

Hardware and software requirements:

H/S requirements

Hardware, only for OneSumX platform components:

- ❑ Portfolio size driven, which can be determined based on data set size
- ❑ Memory expensive application. Off-heap memory is in linear dependency with data volume

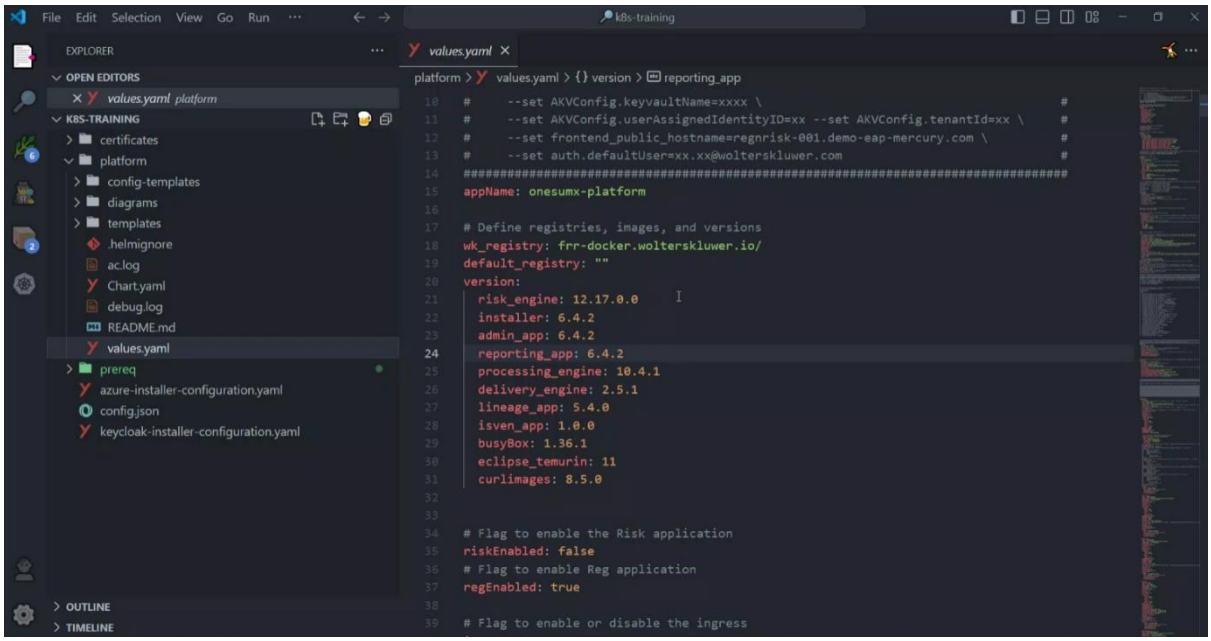
Typical portfolio size – L sized data feed: 1 300 000 financial instruments and 325 000 counterparties:

- 128 GB (premium SSD) disk space
- 16 CPU cores
- 64 + 20 * (# of data feeds) GB of RAM

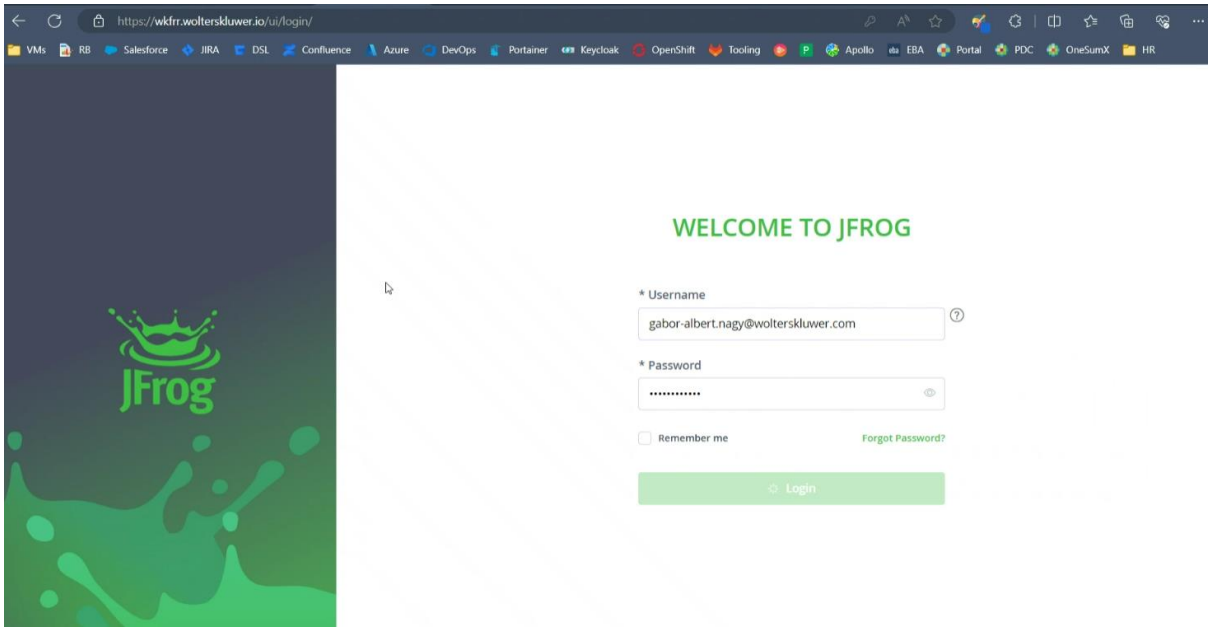
example for 2 parallel datasets --> 64 + 20 * 2 GB of RAM = 104 GB of RAM

Currently the calculation engines cannot distribute the workload across nodes. → The node with calculation engines need to have enough RAM to consume the dataset.

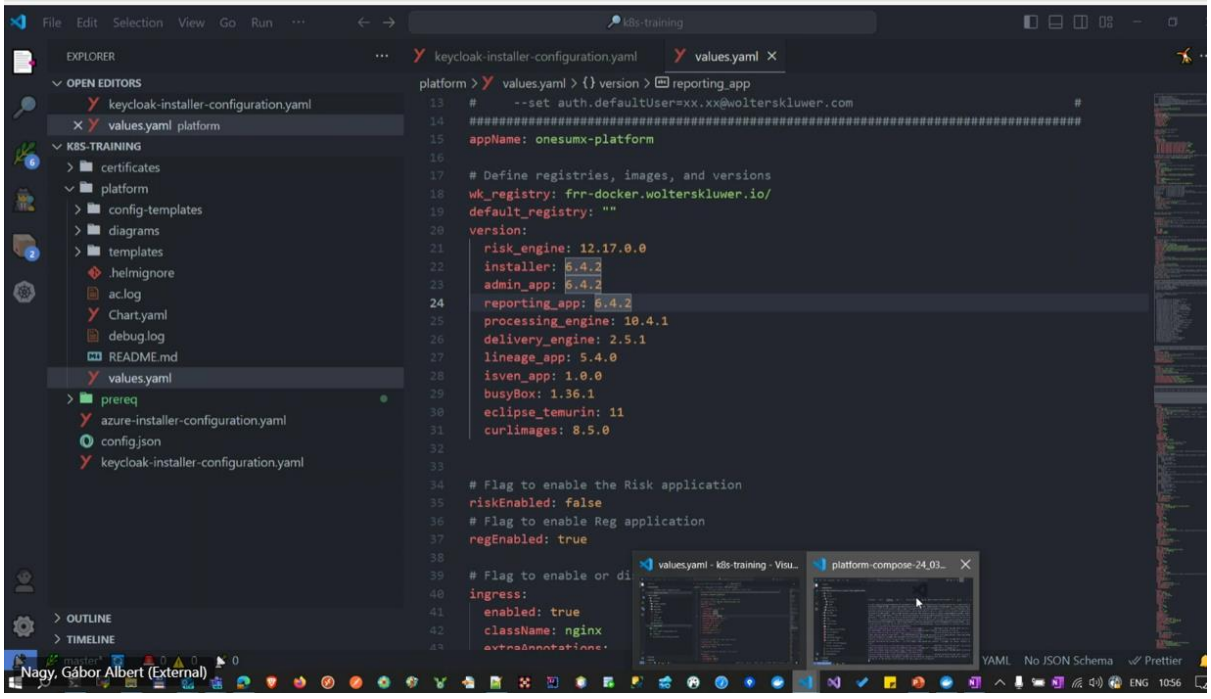
Values.yaml file



JFROG artifactory



Few images used in the deployment from Local repository (w_k registry) and some external repository (default_registry



To enable Risk and/or Regulatory installation we have flags in the configuration file.

```
# Flag to enable the Risk application
riskEnabled: false
# Flag to enable Reg application
regEnabled: true
```

```yaml
platform > values.yaml > riskEnabled

# Flag to enable the Risk application
riskEnabled: false
# Flag to enable Reg application
regEnabled: true

# Flag to enable or disable the ingress
ingress:
  enabled: true
  className: nginx
  extraAnnotations:
    # appgw.ingress.kubernetes.io/request-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-buffering: "on"
    nginx.ingress.kubernetes.io/use-regex: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "2048m"
    nginx.ingress.kubernetes.io/proxy-max-temp-file-size: "2048m"
    nginx.ingress.kubernetes.io/client_max_body_size: "2048m"

# Hostname used to configure redirect between the application. Should be the customer facing
# configuration, usually the DNS name the main ingress answers to.
frontend_public_hostname: ro42gfrrl4656nf.eu.wkglobal.com

 # database configuration
database:
  #Either oracle or sqlserver
  type: mssqlserver
```

Ingress Nginx controller is used in the deployment


```yaml
# Flag to enable or disable the ingress
ingress:
  enabled: true
  className: nginx
  extraAnnotations:
    # appgw.ingress.kubernetes.io/request-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-buffering: "on"
    nginx.ingress.kubernetes.io/use-regex: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "2048m"
    nginx.ingress.kubernetes.io/proxy-max-temp-file-size: "2048m"
    nginx.ingress.kubernetes.io/client_max_body_size: "2048m"

# Hostname used to configure redirect between the application. Should be the customer facing
# configuration, usually the DNS name the main ingress answers to.
frontend_public_hostname: ro42gfrrl4656nf.eu.wkglobal.com

 # database configuration
```


Database configuration Section for mssql or Oracle database

```yaml
# database configuration
database:
  #Either oracle or sqlserver
  type: mssqlserver
  #The section below is only used if the above type is sqlserver
  sqlserver:
    driver: mssqlserver
    driverClass: com.microsoft.sqlserver.jdbc.SQLServerDriver
    host: sqlserver-service
    port: "1433"
    azure: false
    hibernateDialect: org.hibernate.dialect.SQLServer2012Dialect
  #The section below is only used if the above type is oracle
  oracle:
    driver: oracle
    host: ZNLWEDCSEMEPS03.wkfs-frc.local
    port: "1521"
    service: ORACLE
    hibernateDialect: org.hibernate.dialect.Oracle10gDialect
```

Configure the certificate section

```yaml
# If cert manager is disabled, certificates and keystores need to be loaded in a secret called ``onesumx-platform-tls``
# and it must contain the following data elements
# truststore.jks:  a java trustore in jks format
# truststore.p12:  a java trustore in p12 format
# ca.crt:          a certificate for the certificate authority
# keystore.jks:    a java keystore in jks format. This one contains the key for encryption
# keystore.p12:    a java keystore in p12 format
# tls.crt:         a certificate in  crt format
# tls.key:         a private key in pem format
certManager:
  # Enabling cert managers.
  enabled: false
  # A key pair for the certificate authority issuer
  ca_key_pair_secretName: ca-key-pair
  # If selfSigned is set to true then the ca_key_pair_secretName is ignored
  selfSigned: true
  # version: "v1.13.3"
  # namespace: cert-manager
  # installCRDs: true


#use this flag if you want to load additional workspace using configmap
additional_workspace: false
```

Explorer (Ctrl+Shift+E)

platform > values.yaml > {} certManager > enabled

```yaml
  # If cert manager is disabled, certificates and keystores need to be loaded in a secret called ``onesumx-platform-tls``
  # and it must contain the following data elements
  # truststore.jks:  a java trustore in jks format
  # truststore.p12:  a java trustore in p12 format
  # ca.crt:          a certificate for the certificate authority
  # keystore.jks:    a java keystore in jks format. This one contains the key for encryption
  # keystore.p12:    a java keystore in p12 format
  # tls.crt:         a certificate in  crt format
  # tls.key:         a private key in pem format
  certManager:
    # Enabling cert managers.
    enabled: false
    # A key pair for the certificate authority issuer
    ca_key_pair_secretName: ca-key-pair
    # If selfSigned is set to true then the ca_key_pair_secretName is ignored
    selfSigned: true
    # version: "v1.13.3"
    # namespace: cert-manager
    # installCRDs: true


  #use this flag if you want to load additional workspace using configmap
  additional_workspace: false


  # Parameters related to the security context
  securityContext:
    # Setting this to true will force the container filesystem to be readonly. Set to true in production
    setReadOnlyFilesystem: true
    # Setting to true will add the below use configuration to the security context. Setting it to false will ignore the user block below
```

```yaml
# Parameters related to the security context
securityContext:
  # Setting this to true will force the container filesystem to be readonly. Set to true in production
  setReadOnlyFilesystem: true
  # Setting to true will add the below use configuration to the security context. Setting it to false will ignore the user block below
  setUidAndGid: false
  # The user and group to use to run the pods.
  # Pods should share at least the same group to share files from the /usr/workspaces directory.
  user:
    uid: 4201
    gid: 4200
    groupName: onesumx
    userName: onesumx
```

```yaml
#Section to configure open id connect
auth:
  # please make this is -- EXACTLY --the way it is passed from the IDP. Respect the case
  # IT MUST BE THE NAME OF THE ADMIN USER THAT WILL LOG THE FIRST TIME IN THE APPLICATION
  defaultUser: admin
  # The name of the default user
  defaultDisplayedName: "Gabor Nagy"
  idpEntityId: https://keycloak-service/realms/onesumx
  authority: https://keycloak-service/realms/onesumx
  tokenServiceUri: https://keycloak-service/realms/onesumx/protocol/openid-connect/token
  keysUri: https://keycloak-service/realms/onesumx/protocol/openid-connect/certs
  # Leave next fields empty in case implicit endpoints discovery
  authorizationUri:
  endSessionUri:
  userInfoUri:
  registrationId:
  # Leave this one empty on azure
  tokenIntrospectUri: https://keycloak-service/realms/onesumx/protocol/openid-connect/token/introspect
  # Scope required only for azure open id connect. It is based on the application registration name
  scope: openid
  apiId:
  # which filed be user as username
  principalNameKey: preferred_username
#please make the below value as true if you want to use the csi driver for secrets
secretcsi: false
# Use Azure Key Vault or Hashi Corp Vault
# If no Vault is selected, then secrets must be loaded directly as kubernetes secrets.
```

Configure the Components

```yaml
components:
  admin_app:
    # Component name that will be postfixed with "-pod" "-service" "-deploy"
    name: "admin-app"
    image_name: woltersskluwer/onesumx/reg/admin-center
    # Directory in which components specific config files are hosted. usually in the form /usr/config/appConfig/xxx/
    appConfigDir: "admin-app"
    # To add extra label extra labels: { "label1":"val1", "label2":"val2" }
    extraLabels: {}
    # To add extra label extra annotations: { "annotation1":"val1", "annotation2":"val2" }
    extraAnnotations: {}
    replicas: 1
    affinity: {}
    tolerations: []
    database:
      name: SYSTEM_DB
    resources:
      requests:
        cpu: "256m"
        memory: "4Gi"
      limits:
        cpu: "2"
        memory: "6Gi"
    httpPort: 8080
    httpsPort: 8443
```

Configure persistent volumes

```yaml
volumeMounts:
  certificates:
    root: /tmp/usr/certificates
    custom_truststore: /tmp/usr/rootAuthorities
    from_secrets: from_secrets

commonConfigurations:
  pullPolicy: Always
  ONESUMX_RE_AUTOMATIC_USER_PROVISIONING : "false"
  ONESUMX_OAUTH2_MEMBER_OF_KEY : groups
  # Define certificate names that will be mapped to the volume used by the helm chart
  certificate_pfx_path: keystore.p12
  keystore_path: keystore.jks
  truststore_filename: truststore.jks
  keystore_alias: certificate
  certificate_cer_path: regulatory-engine-helm.cer
  certificate_alias: wk-cacerts
  truststore_pwd: changeit
  wolterskluwer_delivery_engine_environment: mercury
```

Configure keystore and DNS

```yaml
keystores:
  jks:
    create: true
    passwordSecretRef:
      key: ssl-keystore-password
      name: {{ include "regulatory-engine.installerConfigurationSecret" . }}
  pkcs12:
    create: true
    passwordSecretRef:
      key: ssl-keystore-password
      name: {{ include "regulatory-engine.installerConfigurationSecret" . }}
usages:
  - server auth
  - client auth
# At least one of a DNS Name, URI, or IP address is required.
dnsNames:
  - processing-engine-service
  - admin-app-service
  - reporting-app-service
  - lineage-app-service
  - risk-engine-service
  - processing-engine-service
  - delivery-engine-service
issuerRef:
  name: certificate-issuer
  kind: Issuer
  group: cert-manager.io
{{- end }}
```

Secret values.yaml

```yaml
  auth:
    endSessionUri:
    userInfoUri:
    registrationId:
    # Leave this one empty on azure
    tokenIntrospectUri: https://keycloak-service/realms/onesumx/protocol/openid-connect/token/introspect
    # Scope required only for azure open id connect. It is based on the application registration name
    scope: openid
    apiId:
    # which filed be user as username
    principalNameKey: preferred_username
  #please make the below value as true if you want to use the csi driver for secrets
  secretcsi: false
  # Use Azure Key Vault or Hashi Corp Vault
  # If no Vault is selected, then secrets must be loaded directly as kubernetes secrets.
  # If useAzureVault is set to true, please refer to the AKVConfig section to configure the vault
  # please refer to section AKVConfig below
  useAzureVault: false
  # If useHashicorpVault is set to true, please refer to the AKVConfig section to configure the vault
  # please refer to section HCVConfig below
  useHashicorpVault: false

  ##########################################################################################
  # Secrets -- Warning some secrets are going to move to config maps (jdbc string and server names) #
  ##########################################################################################
  # Docker container registry credentials. Usually points to the Wolters Kluwer repository
  # $ kubectl -n [namespace]  create secret docker-registry docker-configuration--docker-server=<DOCKER_REGISTRY_SERVER> \
  # --docker-username=<DOCKER_USER> \
  # --docker-password=<DOCKER_PASSWORD> \
  # --docker-email=<DOCKER_EMAIL
```

```yaml
components:
  admin_app:
    # Component name that will be postfixed with "-pod" "-service" "-deploy"
    name: "admin-app"
    image_name: wolterskluwer/onesumx/reg/admin-center
    # Directory in which components specific config files are hosted. usually in the form /usr/config/appConfig/xxx/
    appConfigDir: "admin-app"
    # To add extra label extra labels: { "label1":"val1", "label2":"val2" }
    extraLabels: {}
    # To add extra label extra annotations: { "annotation1":"val1", "annotation2":"val2" }
    extraAnnotations: {}
    replicas: 1
    affinity: {}
    tolerations: []
    database:
      name: SYSTEM_DB
    resources:
      requests:
        cpu: "256m"
        memory: "4Gi"
      limits:
        cpu: "2"
        memory: "6Gi"
    httpPort: 8080
    httpsPort: 8443
    onHeapMinMemory: 4G
    onHeapMemory: 6G
    contextPath: /admin-webapp
```

Docker configuration login

```
S C:\Work\k8s-training\platform> kubectl get pods
AME                                                      READY   STATUS    RESTARTS       AGE
eycloak-7df54b96c-m2gvv                                   1/1     Running   1 (173m ago)   3d14h
ginx-controller-ingress-nginx-controller-7db7f79959-bblnx 1/1     Running   2 (173m ago)   4d12h
qlserver-54cd79f8c9-775dg                                 1/1     Running   1 (173m ago)   3d14h
S C:\Work\k8s-training\platform> kubectl get ep
AME                                          ENDPOINTS                      AGE
eycloak-service                              10.1.2.48:443,10.1.2.48:8080   3d14h
ginx-controller-ingress-nginx-controller     10.1.2.43:443,10.1.2.43:80     4d12h
ginx-controller-ingress-nginx-controller-admission   10.1.2.43:8443         4d12h
qlserver-service                             10.1.2.47:1433                 3d14h
S C:\Work\k8s-training\platform> kubectl get services
AME                                          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                    AGE
eycloak-service                              ClusterIP      10.100.152.76    <none>        8080/TCP,443/TCP           3d14h
ginx-controller-ingress-nginx-controller     LoadBalancer   10.99.170.210    localhost     80:32706/TCP,443:31859/TCP 4d12h
ginx-controller-ingress-nginx-controller-admission   ClusterIP      10.103.201.233   <none>        443/TCP                    4d12h
qlserver-service                             ClusterIP      10.110.234.12    <none>        1433/TCP                   3d14h
S C:\Work\k8s-training\platform> cd ..
S C:\Work\k8s-training> kubectl get secret
AME                                  TYPE                             DATA   AGE
ocker-configuration                  kubernetes.io/dockerconfigjson   1      4d18h
ngress-tls-secret                    Opaque                           2      4d12h
nstaller-configuration               Opaque                           17     3d13h
ginx-controller-ingress-nginx-admission   Opaque                      3      4d13h
nesumx-platform-tls                  Opaque                           7      4d1h
h.helm.release.v1.nginx-controller.v1   helm.sh/release.v1            1      4d12h
h.helm.release.v1.osx-prereq.v1      helm.sh/release.v1               1      3d14h
qlserver-secret                      Opaque                           1      3d14h
S C:\Work\k8s-training>
```

Secret generation of onesumx-platform

```
Windows PowerShell    Windows PowerShell
PS C:\Work\k8s-training\platform> kubectl get pods
NAME                                                      READY   STATUS    RESTARTS       AGE
keycloak-7df54b96c-m2gvv                                  1/1     Running   1 (173m ago)   3d14h
nginx-controller-ingress-nginx-controller-7db7f79959-bblnx 1/1     Running   2 (173m ago)   4d12h
sqlserver-54cd79f8c9-775dg                                1/1     Running   1 (173m ago)   3d14h
PS C:\Work\k8s-training\platform> kubectl get ep
NAME                                          ENDPOINTS                      AGE
keycloak-service                              10.1.2.48:443,10.1.2.48:8080   3d14h
nginx-controller-ingress-nginx-controller     10.1.2.43:443,10.1.2.43:80     4d12h
nginx-controller-ingress-nginx-controller-admission   10.1.2.43:8443         4d12h
sqlserver-service                             10.1.2.47:1433                 3d14h
PS C:\Work\k8s-training\platform> kubectl get services
NAME                                          TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                    AGE
keycloak-service                              ClusterIP      10.100.152.76    <none>        8080/TCP,443/TCP           3d14h
nginx-controller-ingress-nginx-controller     LoadBalancer   10.99.170.210    localhost     80:32706/TCP,443:31859/TCP 4d12h
nginx-controller-ingress-nginx-controller-admission   ClusterIP      10.103.201.233   <none>        443/TCP                    4d12h
sqlserver-service                             ClusterIP      10.110.234.12    <none>        1433/TCP                   3d14h
PS C:\Work\k8s-training\platform> cd ..
PS C:\Work\k8s-training> kubectl get secret
NAME                                  TYPE                             DATA   AGE
docker-configuration                  kubernetes.io/dockerconfigjson   1      4d18h
ingress-tls-secret                    Opaque                           2      4d12h
installer-configuration               Opaque                           17     3d13h
nginx-controller-ingress-nginx-admission   Opaque                      3      4d13h
onesumx-platform-tls                  Opaque                           7      4d1h
sh.helm.release.v1.nginx-controller.v1   helm.sh/release.v1            1      4d12h
sh.helm.release.v1.osx-prereq.v1      helm.sh/release.v1               1      3d14h
sqlserver-secret                      Opaque                           1      3d14h
PS C:\Work\k8s-training> cd ..
PS C:\Work> cd .\k8s-training\certificates\
PS C:\Work\k8s-training\certificates> kubectl create secret generic onesumx-platform-tls --from-file=tls.crt=./tls.crt --from-file=tls.key=./tls.key --from-
file=ca.crt=./ca.crt --from-file=keystore.jks=./keystore.jks --from-file=truststore.jks=./truststore.jks --from-file=truststore.p12=./truststore.p12 --from-
file=keystore.p12=./keystore.p12
```

```
PS C:\Work\k8s-training\platform> kubectl get pods
NAME                                                      READY   STATUS    RESTARTS       AGE
keycloak-7df54b96c-m2gvv                                  1/1     Running   1 (173m ago)   3d14h
nginx-controller-ingress-nginx-controller-7db7f79959-bblnx  1/1     Running   2 (173m ago)   4d12h
sqlserver-54cd79f8c9-775dg                                1/1     Running   1 (173m ago)   3d14h
PS C:\Work\k8s-training\platform> kubectl get ep
NAME                                                ENDPOINTS                      AGE
keycloak-service                                    10.1.2.48:443,10.1.2.48:8080   3d14h
nginx-controller-ingress-nginx-controller           10.1.2.43:443,10.1.2.43:80     4d12h
nginx-controller-ingress-nginx-controller-admission 10.1.2.43:8443                 4d12h
sqlserver-service                                   10.1.2.47:1433                 3d14h
PS C:\Work\k8s-training\platform> kubectl get services
NAME                                                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
keycloak-service                                    ClusterIP      10.100.152.76    <none>        8080/TCP,443/TCP             3d14h
nginx-controller-ingress-nginx-controller           LoadBalancer   10.99.170.210    localhost     80:32706/TCP,443:31859/TCP   4d12h
nginx-controller-ingress-nginx-controller-admission ClusterIP      10.103.201.233   <none>        443/TCP                      4d12h
sqlserver-service                                   ClusterIP      10.110.234.12    <none>        1433/TCP                     3d14h
PS C:\Work\k8s-training\platform> cd ..
PS C:\Work\k8s-training> kubectl get secret
NAME                                  TYPE                                  DATA   AGE
docker-configuration                  kubernetes.io/dockerconfigjson        1      4d18h
ingress-tls-secret                    Opaque                                2      4d12h
installer-configuration               Opaque                                17     3d13h
nginx-controller-ingress-nginx-admission  Opaque                            3      4d13h
onesumx-platform-tls                  Opaque                                7      4d1h
sh.helm.release.v1.nginx-controller.v1  helm.sh/release.v1                  1      4d12h
sh.helm.release.v1.osx-prereq.v1      helm.sh/release.v1                    1      3d14h
sqlserver-secret                      Opaque                                1      3d14h
PS C:\Work\k8s-training> cd ..
PS C:\Work> cd .\k8s-training\certificates\
PS C:\Work\k8s-training\certificates> kubectl create secret generic onesumx-platform-tls --from-file=tls.crt=./tls.crt --from-file=tls.key=./tls.key --from-
file=ca.crt=./ca.crt --from-file=keystore.jks=./keystore.jks --from-file=truststore.jks=./truststore.jks --from-file=truststore.p12=./truststore.p12 --from-
file=keystore.p12=./keystore.p12
error: failed to create secret secrets "onesumx-platform-tls" already exists
PS C:\Work\k8s-training\certificates> kubectl delete secrets onesumx-platform-tls
secret "onesumx-platform-tls" deleted
PS C:\Work\k8s-training\certificates> kubectl create secret generic onesumx-platform-tls --from-file=tls.crt=./tls.crt --from-file=tls.key=./tls.key --from-
file=ca.crt=./ca.crt --from-file=keystore.jks=./keystore.jks --from-file=truststore.jks=./truststore.jks --from-file=truststore.p12=./truststore.p12 --from-
file=keystore.p12=./keystore.p12
secret/onesumx-platform-tls created
PS C:\Work\k8s-training\certificates>
```

Keycloak installer

```
EXPLORER
OPEN EDITORS
  keycloak-installer-configuration.yaml
K8S-TRAINING
  certificates
    ca.crt
    keystore.jks
    keystore.p12
    tls.crt
    tls.key
    truststore.jks
    truststore.p12
  platform
  prereq
  azure-installer-configuration.yaml
  config.json
  keycloak-installer-configuration.yaml
OUTLINE
TIMELINE
```

```yaml
keycloak-installer-configuration.yaml > {} data
data:
  # base64 encoding of the client id for delivery engine
  hub-auth-token-client-id: b25lc3VteA==
  # base64 encoding of the secret for delivery engine
  hub-auth-token-secret: MFBUMjUzWUh2ME5KZktWTDd4UHFzclI4SGEyUEh5bFM=
  # base64 encoding of the client id for lineage app
  lineage-auth-token-client-id: b25lc3VteA==
  # base64 encoding of the secret for lineage app
  lineage-auth-token-secret: MFBUMjUzWUh2ME5KZktWTDd4UHFzclI4SGEyUEh5bFM=
  # base64 encoding of the client id for processing engine
  processing-auth-token-client-id: b25lc3VteA==
   # base64 encoding of the secret for processing engine
  processing-auth-token-secret: MFBUMjUzWUh2ME5KZktWTDd4UHFzclI4SGEyUEh5bFM=
  # base64 encoding of the client id for risk engine
  risk-auth-token-client-id: b25lc3VteA==
   # base64 encoding of the secret for risk engine
  risk-auth-token-secret: MFBUMjUzWUh2ME5KZktWTDd4UHFzclI4SGEyUEh5bFM=
  # User name for system level credentials
  database-admin-username: c2E=
  # SQL Server Admin Password
  database-admin-password: U2VraXJvQDE=
  #  Username for application level credentials
  database-application-username: c2E=
  #  Password for application level credentials
  database-application-password: U2VraXJvQDE=
  # TLS keystore password
  ssl-keystore-password: Y2hhbmdlaXQ=
```

```
KEYSTORE_P12=keystore.p12
TRUSTSTORE_JKS=truststore.jks
TRUSTSTORE_P12=truststore.p12
TLS_CRT=tls.crt
TLS_KEY=tls.key
CA_CRT_OUTPUT=ca.crt
CERT_CSR=cert.csr

KEYSTORE_PWD=""
TRUSTSTORE_PWD=""
CA_KEY_PWD=""

INPUT_TRUSTSTORE=""
USE_EXISTING_KEYSTORE=true
KEY_ALIAS="certificate"
ROOT_CERT=""
CUSTOM_SAN_ATTR=""
CA_CRT=""
CA_KEY=""

DEFAULT_SAN_ATTR="DNS:processing-engine-service,DNS:admin-app-service,DNS:reporting-app-service,DNS:lineage-app-service,DNS:risk-engine-service,DNS:delivery-engine-service"

show_help() {
  echo "Usage: $(basename "$0") [OPTIONS]
  -h | --help                 Display help and exit
  -k | --keystore-password    Password for the server's keystore
  -t | --truststore           (Optional) Path to the truststore in JKS format containing trusted certificates
  -p | --truststore-password  Password for the truststore
  -c | --ca-cert              (Optional) Path to the CA certificate
  -kca | --ca-key             (Optional) Path to the CA private key
  -r | --root-certificates    (Optional) Path to root certificates, defaults to JAVA_HOME/lib/security/cacerts
  -s | --san-attributes       (Optional) Add custom SAN attributes, comma separated, e.g., dns:myhost1,dns:myhost2"
}

parse_args() {
  while [ $# -gt 0 ]; do
    case $1 in
      -k|--keystore-password)
        KEYSTORE_PWD="$2"
        shift 2
        ;;
      -p|--truststore-password)
        TRUSTSTORE_PWD="$2"
        shift 2
        ;;
```

```
keycloak-7df54b96c-m2gvv                                  1/1     Running   1 (173m ago)   3d14h
nginx-controller-ingress-nginx-controller-7db7f79959-bblnx   1/1     Running   2 (173m ago)   4d12h
sqlserver-54cd79f8c9-775dg                                1/1     Running   1 (173m ago)   3d14h
PS C:\Work\k8s-training\platform> kubectl get ep
NAME                                                 ENDPOINTS                      AGE
keycloak-service                                     10.1.2.48:443,10.1.2.48:8080   3d14h
nginx-controller-ingress-nginx-controller            10.1.2.43:443,10.1.2.43:80     4d12h
nginx-controller-ingress-nginx-controller-admission  10.1.2.43:8443                 4d12h
sqlserver-service                                    10.1.2.47:1433                 3d14h
PS C:\Work\k8s-training\platform> kubectl get services
NAME                                                 TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
keycloak-service                                     ClusterIP      10.100.152.76    <none>        8080/TCP,443/TCP             3d14h
nginx-controller-ingress-nginx-controller            LoadBalancer   10.99.170.210    localhost     80:32706/TCP,443:31859/TCP   4d12h
nginx-controller-ingress-nginx-controller-admission  ClusterIP      10.103.201.233   <none>        443/TCP                      4d12h
sqlserver-service                                    ClusterIP      10.110.234.12    <none>        1433/TCP                     3d14h
PS C:\Work\k8s-training\platform> cd ..
PS C:\Work\k8s-training> kubectl get secret
NAME                                                 TYPE                                  DATA   AGE
docker-configuration                                 kubernetes.io/dockerconfigjson        1      4d18h
ingress-tls-secret                                   Opaque                                2      4d12h
installer-configuration                              Opaque                                17     3d13h
nginx-controller-ingress-nginx-admission             Opaque                                3      4d13h
onesumx-platform-tls                                 Opaque                                7      4d1h
sh.helm.release.v1.nginx-controller.v1               helm.sh/release.v1                    1      4d12h
sh.helm.release.v1.osx-prereq.v1                     helm.sh/release.v1                    1      3d14h
sqlserver-secret                                     Opaque                                1      3d14h
PS C:\Work\k8s-training> cd ..
PS C:\Work> cd .\k8s-training\certificates\
PS C:\Work\k8s-training\certificates> kubectl create secret generic onesumx-platform-tls --from-file=tls.crt=./tls.crt --from-file=tls.key=./tls.key --from-
file=ca.crt=./ca.crt --from-file=keystore.jks=./keystore.jks --from-file=truststore.jks=./truststore.jks --from-file=truststore.p12=./truststore.p12 --from-
file=keystore.p12=./keystore.p12
error: failed to create secret secrets "onesumx-platform-tls" already exists
PS C:\Work\k8s-training\certificates> kubectl delete secrets onesumx-platform-tls
secret "onesumx-platform-tls" deleted
PS C:\Work\k8s-training\certificates> kubectl create secret generic onesumx-platform-tls --from-file=tls.crt=./tls.crt --from-file=tls.key=./tls.key --from-
file=ca.crt=./ca.crt --from-file=keystore.jks=./keystore.jks --from-file=truststore.jks=./truststore.jks --from-file=truststore.p12=./truststore.p12 --from-
file=keystore.p12=./keystore.p12
secret/onesumx-platform-tls created
PS C:\Work\k8s-training\certificates> cd ..
PS C:\Work\k8s-training> kubectl apply -f .\keycloak-installer-configuration.yaml
secret/installer-configuration unchanged
PS C:\Work\k8s-training>
```

Generate the keycloak client secrete

```
Context: docker-desktop                    <0> tail   <6> 1h   <shift-c> Clear          <t> Toggle Timestamp
Cluster: docker-desktop                    <1> head            <c>       Copy           <w> Toggle Wrap
User:    docker-desktop                    <2> 1m              <m>       Mark
K9s Rev: v0.28.2 ⚡v0.32.4                 <3> 5m              <ctrl-s>  Save
K8s Rev: v1.25.0                           <4> 15m             <s>       Toggle AutoScroll
CPU:     n/a                               <5> 30m             <f>       Toggle FullScreen
MEM:     n/a

Logs(mercury-n/admin-app-installer-job-m6d4w:installer)[5m]
Autoscroll:On    FullScreen:Off    Timestamps:Off    Wrap:Off
tee: /tmp/usr/logs/installer/RE_installer_2024-06-14T084326.log: Permission denied
The installer will create only the database
/usr/config/install.properties found.
Skip certificate generation
Start creating system database
/usr/lib/jvm/java-17-amazon-corretto
-Xms1G -Xmx3G -XX:MetaspaceSize=1G -server -XX:+AlwaysPreTouch -XX:+UseG1GC -XX:+ScavengeBeforeFullGC -XX:+DisableExplicitGC -XX:+DisableExplicitGC -Djb
log4j:WARN No appenders could be found for logger (com.wkfsfrc.platform.shell.installer.DbScriptExecutor).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Done creating system database
Skip data sources generation
Start running liquibase
Jun 14, 2024 8:43:27 AM liquibase.servicelocator
INFO: Cannot load service: liquibase.license.LicenseService: Provider liquibase.license.pro.DaticalTrueLicenseService could not be instantiated
##################################################
##                                              ##
##  Get documentation at docs.liquibase.com     ##
##  Get certified courses at learn.liquibase.com ##
##  Free schema change activity reports at      ##
##      https://hub.liquibase.com               ##
##                                              ##
<pod> <containers> <logs>
```

OpenShift:

RedHat OpenShift

Origin: Kubernetes is an **open-source project** managed by the Cloud Native Computing Foundation (CNCF). OpenShift, on the other hand, is a product offered by Red Hat, built **on top of Kubernetes**. It includes additional features and tools to **enhance the developer and operations experience.**

Ease of Use: Kubernetes is highly configurable and extensible, but it can be complex to set up and manage. OpenShift provides a more **streamlined and opinionated experience**, with built-in defaults and helpers to simplify configuration and management.

Enhanced Security: OpenShift runs containers as **a non-root user** by default and enforces. **Security Context Constraints (SCC),** which can provide a more secure environment than a default Kubernetes installation.

UI: OpenShift includes a comprehensive and user-friendly web console for managing applications and resources, in addition to the command-line interface. Kubernetes also has a dashboard, but it's less feature-rich compared to OpenShift's.

Pricing: Kubernetes is free and open-source. You only pay for the infrastructure it runs on. OpenShift, however, comes in several paid editions (OpenShift Container Platform, OpenShift Dedicated, OpenShift Online) that provide additional support and features.

Enterprise Support: OpenShift is backed by Red Hat, which offers enterprise-level support. This can be a significant advantage for businesses that need guaranteed support and uptime.

Multi-Tenancy Support : OpenShift provides strong multi-tenancy support with features like project and user management, which can be beneficial for large teams and organizations.

Kubernetes vs RedHat OpenShift

		OPENSHIFT
Originally developed by Google, now maintained by the Cloud Native Computing Fundation		Developed by RedHat
Apache License 2.0		Exists in Open Source : OpenShift origin (Apache License 2.0, and an enterprise version : RH OpenShift
Limited UI, limited to a dashboard	Vs	Extensive UI and very use case oriented (CI/CD)
~4 updates a year		~ 3 updates a year
Very flexible and extensible		Is a superset of the K8S features but very opinionated in their usage
Very large community and eco-system		Support provided by RedHat

RedHat OpenShift Attention Points

Project (OC) vs namespace (K8S) : These are the same, just a name difference.

Rootless containers in OpenShift : The Helm Charts explicitly set `allowPrivilegeEscalation: false`

SCC - random UID allocated for files. Cannot rely on UIDs for scripting

OC Container file system is read-only (except /tmp and mounted volumes) : The Helm Charts explicitly set `readOnlyRootFilesystem:true`

Routers (OC) vs Ingress (K8S)

Helm Charts 24.03.07

Helm Charts can be downloaded from the registry:

```bash
# Login into the repo
$ helm repo add wkfrr-helm https://wkfrr.wolterskluwer.io/artifactory/api/helm/wkfrr-helm --
username <username> --password <TOKEN_from salesforce>

# Download the charts
$ helm pull wkfrr-helm/platform --version 24.03.07

# Upload the charts
$ helm repo update
```

Reach out at

FRR_Questions_Answered@wolterskluwer.com

Questions:

Q: Do we need to provide k8s for customers?

A: Most of the customers have k8s cluster setup

Q: Any monitoring tools?

A: No.